

Zameen Scraper Evaluation Report

GitHub Repository:

[View the Source Code on GitHub](#)

PART 1: BEFORE REFACTORING

1. Modularity

Are the modules highly cohesive?

FAIL

The main orchestrator function was doing a lot of heavy lifting by managing thread pools, orchestrating phases, and handling file I/O directly.

Is the coupling between modules low?

FAIL

The URL scraping function was tightly coupled to shared state (lists and locks), which it mutated directly rather than returning values for the caller to handle.

Does each module hide its internal implementation details?

PASS

Functions handled their own specific domains well.

Are interfaces between modules well-defined, simple, and stable?

PASS

Function signatures and type hints were mostly clear.

Can individual modules be tested independently of the rest of the system?

FAIL

The property features extraction made active HTTP requests inside the function, making it difficult to test offline.

Can you replace or upgrade one module without needing to modify others?

PASS

For example, the function locating the hydrated state could be swapped easily.

2. Readability

Are variable, function, and class names descriptive, clear, and intention-revealing?

PASS

Variables and function names were excellent.

Is the code formatting consistent according to standard style guides?

PASS

Standard formatting practices were mostly respected.

Are complex expressions broken down into smaller, well-named intermediate variables or functions?

PASS

Complex dictionary navigations were broken down.

Are comments used to explain *why* a decision was made, rather than restating *what* the code does?

FAIL

Many comments only stated what the code was doing, which was already obvious from the code itself.

Are functions short, fitting on a single screen, and focused on a single task?

FAIL

The property feature extraction was over 70 lines long and handled multiple responsibilities. The category scraping was also quite long.

Is the logic flow straightforward, avoiding deep nesting and overly complex conditionals?

FAIL

The slug gathering loop had deep "arrow code" nesting (multiple nested loops and conditions).

Are standard idioms and patterns of the programming language followed?

PASS

Standard idioms were followed.

3. Maintainability

Is the codebase well-documented at a high level?

FAIL

There was no module-level documentation explaining the architecture, the scraping phases, or the concurrency model.

Are there automated tests (unit, integration) that verify functionality?

FAIL

No tests existed.

Is the code structured to minimize the "blast radius" of changes?

PASS

Changes in extraction logic did not break the crawling orchestrator.

Are magic numbers and hardcoded strings extracted into named constants or configuration files?

FAIL

While top-level settings were constants, there were magic numbers inside functions like timeouts and specific network error codes.

Is duplicated logic avoided by extracting common functionality?

FAIL

Network sessions were initialized in multiple places, making it hard to globally change session settings later.

Are error handling and logging implemented robustly and consistently?

FAIL

The script used raw terminal print statements instead of built-in logging modules.

Does the design follow established object-oriented or functional design principles?

FAIL

Procedural design passed locks and lists around instead of encapsulating them in a class.

PART 2: AFTER REFACTORING

1. Modularity

Are the modules highly cohesive?

PASS

Perfectly encapsulated. The `ZameenScraper` and `ZameenParser` classes clearly define responsibilities.

Is the coupling between modules low?

PASS

`ZameenScraper` instance encapsulates all shared threading state, completely decoupling the parser logic.

Does each module hide its internal implementation details?

PASS

Classes further improve encapsulation.

Are interfaces between modules well-defined, simple, and stable?

PASS

Clear, typed function and method signatures are maintained.

Can individual modules be tested independently of the rest of the system?

PASS

`ZameenParser` now operates purely on string/soup inputs, making it fully unit-testable offline without HTTP requests.

Can you replace or upgrade one module without needing to modify others?

PASS

Extraction logic can be swapped easily by just modifying static methods.

2. Readability

Are variable, function, and class names descriptive, clear, and intention-revealing?

PASS

Descriptive naming conventions have been maintained.

Is the code formatting consistent according to standard style guides?

PASS

Standard PEP 8 practices are respected.

Are complex expressions broken down into smaller, well-named intermediate variables or functions?

PASS

Complex navigations are successfully broken down.

Are comments used to explain *why* a decision was made, rather than restating *what* the code does?

PASS

Comments now explain the reasons behind design decisions (e.g., impersonating Chrome to bypass WAF).

Are functions short, fitting on a single screen, and focused on a single task?

PASS

Functions were broken down into `_extract_location`, `_extract_basic_info`, etc., drastically reducing size.

Is the logic flow straightforward, avoiding deep nesting and overly complex conditionals?

PASS

Nesting was flattened by extracting the page processing logic into the `_process_search_page` method.

Are standard idioms and patterns of the programming language followed?

PASS

Standard idioms are followed consistently.

3. Maintainability

Is the codebase well-documented at a high level?

PASS

Classes include descriptive docstrings explaining their architectural role and overall purpose.

Are there automated tests (unit, integration) that verify functionality?

FAIL

No tests are provided yet (though the refactored code makes testing trivially easy).

Is the code structured to minimize the "blast radius" of changes?

PASS

Extraction logic updates will not affect the crawling orchestration engine.

Are magic numbers and hardcoded strings extracted into named constants or configuration files?

PASS

Moved to uppercase constants at the top of the file (e.g., MAX_RETRIES, HTTP_TIMEOUT).

Is duplicated logic avoided by extracting common functionality?

PASS

Network sessions and locking mechanisms are managed centrally by the class.

Are error handling and logging implemented robustly and consistently?

PASS

Standard Python logging module is now utilized for all tracing and error reporting.

Does the design follow established object-oriented or functional design principles?

PASS

Object-Oriented design is adopted, completely separating Parsing logic from Scraping state.